

## StatAverageTraveledDistance [SimTalk] - WorkerPool

Returns the average distance in meters which the *Worker* traveled from the *WorkerPool* designated by <Path> to the *Workplaces* attached to the stations and between the *stations*.

### Type

Read-only attribute

### Syntax

```
<Path>.StatAverageTraveledDistance → length
```

### Return Value

The return value has the data type *length*.

### Example

```
print MyWorkerPool.StatAverageTraveledDistance
```

## Attributes of the WorkerPool

### \_Attributes of the WorkerPool

The *WorkerPool* provides:

- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

| Name             | Value     | Inherited | Watchable | Signature |
|------------------|-----------|-----------|-----------|-----------|
| MoveToFolderCtrl | VOID      | ✓         |           | method    |
| MTTR             | 0.0000    |           |           | time      |
| Name             | "Station" | ✓         | *         | string    |

Name of the attribute      Current value of the attribute      Data type of the assignment value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyWorkerPool.BrokerPath := mybroker
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyWorkerPool.BrokerPath  
posit := Station.Cont.XPos
```

## AdditionalServices [SimTalk]

Sets the additional services, which the *Worker* provides, in the **Workers to Create table** of the *WorkerPool* designated by <Path>.

### Remarks

*Plant Simulation* addresses the attribute *AdditionalServices* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).AdditionalServices:string[]
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create table** in which you are going to set the attribute.
- The parameter *AdditionalServices* is an *array* of data type *string* which contains the additional services that the *Worker* can execute.

### Assignment Value

You can assign a value of data type *string*.

### Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).AdditionalServices := ["drill1",  
"drill2", "drill3"]
```

### SimTalk

#### [getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

#### [Workers to Create](#)

## Amount [SimTalk] - WorkerPool

Sets how many *Workers* the **Workers to Create** table of the *WorkerPool* designated by <Path> creates.

### Remarks

*Plant Simulation* addresses the attribute *Amount* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Amount:integer
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Amount* of data type *integer* sets the number of *Workers* that is going to be created.

### Assignment Value

You can assign a value of data type *integer*.

### Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).Amount := 2
```

### SimTalk

[getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

[Workers to Create](#)

## BrokerPath [SimTalk] - WorkerPool

Sets the path to the *Broker*, which the *WorkerPool* designated by <Path> uses to provide the services.

### Type

Attribute

### Syntax

```
<Path>.BrokerPath:object
```

### Assignment Value

You can assign a value of data type *object*.

### Example

```
print MyWorkerPool.BrokerPath  
MyWorkerPool.BrokerPath := myBroker  
MyWorkerPool.BrokerPath := Broker3
```

See also

[Broker \[WorkerPool\]](#)

## Efficiency [SimTalk] - WorkerPool

Sets the efficiency of *Workers* which the **Workers to Create** table of the *WorkerPool* designated by <Path> creates.

### Remarks

*Plant Simulation* addresses the attribute *Efficiency* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Efficiency:real
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Efficiency* of data type *real* sets how fast the *Worker* executes the jobs.
  - With an **Efficiency** of 100 percent the *Worker* consumes the exact processing time which you specified.
  - With an **Efficiency** of 200 percent he consumes half of the processing time which you specified.
  - With an **Efficiency** of 50 percent he consumes twice the processing time which you specified.

### Assignment Value

You can assign a value of data type *real*.

### Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).Efficiency := 75
```

### SimTalk

#### [getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

[Workers to Create](#)

## EntranceCtrl [SimTalk] - WorkerPool

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* as soon as the *Worker* has entered the *WorkerPool* designated by <Path>.

### Type

Attribute

### Syntax

```
<Path>.EntranceCtrl:method
```

### Assignment Value

You can assign a value of data type *method*.

### Example

```
MyWorkerPool.EntranceCtrl := &myEntranceCtrl
```

### See also

[Entrance Control \[WorkerPool\]](#)



## ExitCtrl [SimTalk] - WorkerPool

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* as soon as the *Worker* exits the *WorkerPool*.

### Type

Attribute

### Syntax

```
<Path>.ExitCtrl:method
```

### Assignment Value

You can assign a value of data type *method*.

### Example

```
MyWorkerPool.ExitCtrl := &myExitControl
```

### See also

[Exit Control \[WorkerPool\]](#)

## GetJobOrdersAtHomeOnly [SimTalk]

Sets if the *Worker* gets new job orders for performing a service at a station in the *WorkerPool* designated by <Path> or at the **Home Location** only (`true`).

### Remarks

Specify `false` if you want to assign new job orders to the *Worker* anywhere, no matter if he walks freely in the area or if he walks on a *FootPath*.

A *Worker*, who is staying on a *Workplace*, for which the setting **Worker Stays Here After Completing the Job** is activated, cannot be brokered in this case. You thus have to clear **Worker Stays Here After Completing the Job** in all *Workplaces* so that brokering the *Workers* works correctly.

### Type

Attribute

### Syntax

```
<Path>.GetJobOrdersAtHomeOnly: boolean
```

### Assignment Value

You can assign a value of data type *boolean*.

### Example

```
MyWorkerPool.GetJobOrdersAtHomeOnly := false
```

### SimTalk

[WorkerStaysHere \[SimTalk\]](#)

[HomeLocation \[SimTalk\] - Worker](#)

See also

[Get Job Orders At Home Only](#)

[Worker Stays Here After Completing the Job](#)

## HomeLocation [SimTalk] - WorkerPool

Sets the path to the **Home Location**, i.e., the *Workplace* to which the *Worker* walks, once he has finished his current job, in the **Workers to Create table** of the *WorkerPool* designated by <Path>.

### Remarks

*Plant Simulation* transfers *Workers*, for whom you defined a **Home Location** and a **Shift** in the **Workers to Create table** of the *WorkerPool*, to the defined **Home Location** at the start of the **Shift**. They then start walking to the respective *Workplace* from there.

At the end of the shift *Plant Simulation* transfers all *Workers* of this shift directly to the *WorkerPool*, they do not walk back to the *WorkerPool*.

*Plant Simulation* transfers *Workers*, for whom you defined a **Home Location** but no **Shift**, to the defined **Home Location** at the start of the simulation.

### Note

*Plant Simulation* addresses the attribute *HomeLocation* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Note

*Plant Simulation* always resolves **relative paths** in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(row:integer).HomeLocation:path
```

### Assignment Value

You can assign a value of data type *object*.

### Example

```
WorkerPool.getWorkersToCreateTableRow(1).HomeLocation := HomeLocationWP
print WorkerPool.getWorkersToCreateTableRow(1).HomeLocation // returns
HomeLocationWP
```

## SimTalk

### [getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

## See also

### [Workers to Create](#)

### [The Relative Path](#)

## InitCtrl [SimTalk] - WorkerPool

Designates a *Method* object of the object designated by <Path>.

### Remarks

*Plant Simulation* calls the *Method* once at the beginning of the simulation run during the *init* phase for the *WorkerPool* designated by <Path> before the objects are initialized, and before *init* methods are executed.

You can also initialize attributes which affect the generation of events, such as the **Availability**. To accomplish this the init controls are executed before the events are computed. The normal *init methods* are executed after the initial events have been executed.

Within the **Init Control** you can, for example, change the **Workers to Create table** of the *WorkerPool* for the next simulation run. This cannot be accomplished in an *init method* as this method would be called too late in time.

### Type

Attribute

### Syntax

```
<Path>.InitCtrl:method
```

### Assignment Value

You can assign a value of data type *method*.

### Example

```
MyWorkerPool.InitCtrl := &myInitControl
```

### SimTalk

[init \[SimTalk\] - predefined name](#)

See also

[Init Control \[WorkerPool\]](#)

[Workers to Create](#)

## PartsBuffer [SimTalk]

Sets the path to the object into which the *Worker* deposits the parts he could not deliver, when he walks back to the *WorkerPool* designated by <Path> at the end of his shift.

### Remarks

The *parts buffer* can be any one of the material flow objects, which accepts parts. If you did not assign a parts buffer to the *WorkerPool*, and a *Worker* arrives with parts at the end of its shift in the *WorkerPool*, *Plant Simulation* shows a warning and stops the simulation.

If you activated the *transport importer* in the *parts buffer*, one of the available *Workers* carries the parts in the *parts buffer* to the destination object, after the next shift has started. If you did not activate the *transport importer*, the parts in the *parts buffer* are moved on according to the **Exit Strategy** which you selected for the *parts buffer*.

### Type

Attribute

### Syntax

```
<Path>.PartsBuffer : object
```

### Assignment Value

You can assign a value of data type *object*.

### Example

```
print Myworkerpool.PartsBuffer  
MyWorkerPool.PartsBuffer := MyPartsBuffer
```

### See also

### Parts Buffer

## Scope [SimTalk] - WorkerPool

Sets the path to the material flow objects to which the *Worker* can be brokered in the **Workers to Create** table of the *WorkerPool* designated by <Path>. The scope also encompasses *sub-Frames*.

### Remarks

*Plant Simulation* addresses the attribute *Scope* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Scope:string[]
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Scope* is an *array* of data type *string* which contains the paths of the objects within the scope separated by semicolons.

### Assignment Value

You can assign an *array* of data type *string*.

### Examples

```
// gets the scope
var Scope:object[] := MyWorkerPool.getWorkersToCreateTableRow(1).Scope
```

```
// sets the scope
MyWorkerPool.getWorkersToCreateTableRow(1).Scope := [ path1, path2, ... ] //
assigns the scope
MyWorkerPool.getWorkersToCreateTableRow(1).Scope := [] // deletes the
scope
MyWorkerPool.getWorkersToCreateTableRow(1).Scope := void // deletes the
scope
MyWorkerPool.getWorkersToCreateTableRow(1).Scope := path // allows the
scope to contain a path
```

```
// accesses a column in the Workers to Create table
var t:table := MyWorkerPool.getWorkersToCreateTableRow()
print t["Scope", 1] //results in paths separated by semicolons
var Scope: string[] := splitString(t["Scope", 1], ";") // gets and splits
the paths containing semicolons
print Scope // prints an array of paths

t["Scope", 1] = Scope.join(";")
```

SimTalk

[getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

[Workers to Create](#)



## Shift [SimTalk] - WorkerPool

Sets the shift, during which the *Workers* work, in the **Workers to Create table** of the *WorkerPool* designated by <Path>.

### Remarks

*Plant Simulation* addresses the attribute *Shift* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Shift:string
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Shift* of data type *string* designates the shift during which the *Worker* works. Define the shift in the assigned *ShiftCalendar*.

### Assignment Value

You can assign a value of data type *string*.

### Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).Efficiency := "My Shift"
```

### SimTalk

[getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

[Workers to Create](#)

## Speed [SimTalk] - WorkerPool

Sets the speed, with which the *Workers* in the model walk, in the **Workers to Create** table of the *WorkerPool* designated by <Path>.

### Remarks

*Plant Simulation* addresses the attribute *Speed* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Speed:speed
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Speed* of data type *speed* designates the speed with which the *Workers* walk on *FootPaths*.

### Assignment Value

You can assign a value of data type *speed*.

#### Note

In *SimTalk* 2.0 you can specify the speed units mps, fps, kmh, and mph.

Type in the unit directly after the number, without a separating blank space, for example **100kmh** or **100.5kmh**.

```
var len := 1.0ft  
var s : speed := 10.5m / 1:30  
var x : length := 3m
```

## Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).Speed := 3.14
```

## SimTalk

[getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

## See also

[Workers to Create](#)

## Worker [SimTalk] - WorkerPool

Sets the class of the *Workers* which the **Workers to Create** table of the *WorkerPool* designated by <Path> creates.

### Remarks

*Plant Simulation* addresses the attribute *Worker* as sub-attribute of the method *getWorkersToCreateTableRow*.

### Type

Sub-attribute

### Syntax

```
<Path>.getWorkersToCreateTableRow(Row:integer).Worker:object
```

### Parameters

You can specify the following parameters:

- The parameter *Row* of data type *integer* designates the row in the **Workers to Create** table in which you are going to set the attribute.
- The parameter *Worker* of data type *object* sets the class of the *Workers* which the *WorkerPool* uses as a template for creating *Workers*

### Assignment Value

You can assign a value of data type *object*.

### Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).Worker := .UserObjects.MyWorker
```

### SimTalk

#### [getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

#### [Workers to Create](#)

## WorkersCanWorkRemotely [SimTalk]

Sets if the *Worker* managed by the *WorkerPool* designated by <Path> will do the job from his current location, when no free *Workplace* is available at the station to which he was assigned (*true*) or not (*false*).

### Remarks

#### Note

You can use *WorkersCanWorkRemotely* to build a model, which does not contain any *Workplaces*. The *Worker* will then stay in the *WorkerPool* the entire time and work from there.

#### Note

When a free *Workplace* is available, the *Worker* will use this *Workplace*, even if you set *WorkersCanWorkRemotely* to *true*. If you want to prevent a certain service from using a *Workplace*, you can either use an *Exporter* or restrict the *Workplaces* to certain services.

### Type

Attribute

### Syntax

```
<Path>.WorkersCanWorkRemotely : boolean
```

### Assignment Value

You can assign a value of data type *boolean*.

### Example

```
MyWorkerPool.WorkersCanWorkRemotely := false
```

### See also

### Workers Can Work Remotely

## WorkersTravelMode [SimTalk]

Sets how the *Worker* who is managed by the *WorkerPool* designated by <Path> moves in the simulation model.

### Remarks

The *WorkersTravelMode* will only be transmitted from the *WorkerPool* to the *Workers* when they are created, meaning that it will not be changed after the *Workers* are created.

### Type

Attribute

### Syntax

```
<Path>.WorkersTravelMode:string
```

### Assignment Value

You can assign a value of data type *string*.

You can specify:

- "Move freely within area"

The *Worker* walks freely within the area of the simulation model and walks around obstacles, which you defined in 3D.

- "Walk along footpaths"

The *Worker* walks on the *Footpaths*, which you inserted between the *Stations/Workplaces*, to the station at which he is going to work.

- "Beam to workplace"

The *Worker* is beamed (teleported) to the *Workplace* to which he was brokered and at which he is going to work instantly, without using up any time, provided he cannot get to this *Workplace* on any *FootPath*.

#### Note

You can use the feature **Beam to workplace** to build a model, in which you do not want to simulate the *Worker* walking. The *Worker* then always jumps between the *WorkerPool* and the *Workplaces*.

### Example

```
MyWorkerPool.WorkersTravelMode := "Move freely within area"
```

See also

[Travel Mode \[drop-down list\]](#) > [Move freely within area](#)

[Travel Mode \[drop-down list\]](#) > [Walk along footpaths](#)

[Travel Mode \[drop-down list\]](#) > [Beam to workplace](#)

## Worker

### Worker [object]

Use the object *Worker* for doing a job on a *Workplace* at a station.

#### Image



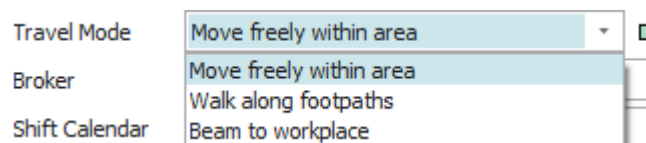
#### Description

The *Worker*-[WorkerPool](#)-[Workplace](#)-[FootPath](#)-concept refines the [Broker](#)-[Importer](#)-[Exporter](#)-concept.

As opposed to the *Exporter*, the *Worker* consumes **time** when **walking** to the *Workplace* at which he does his job. The *Worker* has the same statistics values as the *Exporter*, and in addition provides a statistics value for being **En-route to the Job**.

If your *Worker* does not walk, make sure that the importer on the tab [Importer](#) of the station is activated, on whose *Workplace* the *Worker* is to work.

In the [WorkerPool](#) you can select the [Travel Mode](#) with which the *Worker* gets to his work station:



- [Move freely within area](#)

The *Worker* walks freely within the area of the simulation model and walks around [obstacles](#), which you defined in 3D.

[Model a Worker Who Walks Freely Within the Model](#) gives a short introduction to the topic.

- [Walk along footpaths](#)

The *Worker* walks on the *Footpaths*, which you inserted between the *Stations/Workplaces*, to the station at which he is going to work.